

CS 3364 Algorithms - Project 1 Report

1. Problem interpretation

Five files each provide a rank for the same 10,000 web pages. Line position identifies the page across files. For each page, the program adds its five ranks to form an equally weighted combined rank. A source is considered more reliable when its order has fewer pairwise disagreements, or inversions, with the combined order.

After arranging pages by combined rank, a source produces a sequence of its own ranks. An inversion is a pair of positions $i < j$ for which $\text{sequence}[i] > \text{sequence}[j]$. For example, $[4, 5, 8, 9, 6]$ has two inversions: $(8, 6)$ and $(9, 6)$.

The input contains ties in combined rank. Pages tied in the combined rank are ordered by the source's own rank when building that source's sequence, so a combined-rank tie cannot create an inversion. Equal source ranks also do not form an inversion. This tie policy is part of the program's definition of disagreement and makes the reported counts reproducible.

2. Methodology and design

The program reads all five files, checks that they contain the same number of entries, and sums ranks by page position. It then builds one sequence per source in combined-rank order. Each of three modified sorting algorithms receives the same sequence and returns both a sorted copy and an inversion count:

Algorithm	How inversions are counted	Time complexity
Merge sort	When a value from the right half is placed before remaining values in the left half, each remaining left value forms an inversion with it.	$\Theta(n \log n)$ in all cases
Quicksort	A stable three-way partition preserves order within the less, equal, and greater groups. As each value arrives, earlier values in larger groups contribute cross-partition inversions; recursion counts inversions within groups. The pivot is selected with a fixed-seed random generator.	Expected $\Theta(n \log n)$; worst-case $\Theta(n^2)$
Binary insertion sort	A binary search finds the position after all equal values in the sorted prefix. Every larger value after that position came earlier in the input and forms one inversion.	Worst-case $\Theta(n^2)$ because list insertion moves values

The code checks each algorithm's sorted output against Python's `sorted` and requires all three inversion counts to agree for each source. These checks catch many implementation errors, although agreement alone is not an independent proof of full-input correctness.

3. Experiments and results

The experiment used the five supplied files, each containing 10,000 ranks. It ran under Python 3.12.4 on macOS 26.6.2 (Apple Silicon). The reported time is the median of five runs over all five source sequences at a given length. For the smaller sizes, each sequence was truncated to its first 1,000 or 5,000 values after arranging pages by combined rank. Timing includes each algorithm's sorting and the `sorted` output check, but excludes file loading and sequence construction. Milliseconds vary by machine and run.

Values per source	Merge sort, five sources	Quicksort, five sources	Binary insertion sort, five sources
1,000	4.012 ms	3.093 ms	2.234 ms
5,000	22.937 ms	16.785 ms	19.024 ms
10,000	49.204 ms	34.472 ms	48.914 ms

All three algorithms returned the same inversion counts on the full data:

Reliability order	Source	Inversions
1	source1	17,442,716
2	source5	17,458,031
3	source3	17,502,205
4	source2	17,576,151
5	source4	17,778,159

The program's tests passed for the assignment example, sorted and reverse-sorted lists, duplicate values, 300 short random lists checked by a brute-force pair counter, input preservation, and a 3,000-value rise-then-fall sequence. As a separate check of the full inputs, an independent Fenwick-tree counter produced the same five counts shown above. That check was run for validation; it is not part of the submitted sorting program.

4. Conclusions

Under the stated tie policy and equal-weight combined ranking, source1 is the most reliable of the five sources because it has the fewest inversions. Its count is 15,315 lower than source5, the next closest source. Quicksort was fastest at 10,000 values in these measurements, while merge sort has a guaranteed $\Theta(n \log n)$ running time. Binary insertion sort was competitive on this input despite quadratic worst-case movement cost; that observation does not change its asymptotic bound.

This reliability measure compares each source with an aggregate that includes that source's own ranks. It measures agreement with the combined order, not external search-result quality. Future work could compare other tie rules, independent relevance judgments, and larger or deliberately adverse inputs.

5. Team contributions

Contributions to be confirmed by the team before submission.